Decorate Graphs with Plot Styles and Types Transcript

Hello everyone, welcome to BITS PLANET DIGITAL. Today we will continue our learning journey for the course Data Visualisation Storytelling. We will be now exploring additional features in Matplotlib as part of our module for using Python and other softwares for building effective visualisations.

As in the last class, we will be now doing a live demo session using the Colab framework for understanding the various features of Matplotlib library. So I will be simultaneously running this code in a new Colab file so that you can understand how the functioning of each of the code snippet is there. So in today's class, we will be understanding how to use Matplotlib features for decorating graphs or improving the pre-attentive attributes in your graphics.

So first and foremost, we will try to understand how to use colours. If you recall, colour is one of the strategic element in the pre-attentive attribute that can be leveraged for improving the effectiveness of visualisations. So let's go to the source documentation first and then see how to refer the usage of colour.

So in the example section of Matplotlib, you have a section called colour in which you have different examples by which you can understand the use of colour in Matplotlib. So now let us go to the content. So as in case of any Python exercise, you have to first import the necessary libraries.

I am importing the necessary libraries. My first is NumPy, which is a numerical library which helps us to do the arithmetic calculations. Then you have Matplotlib, the plotting software.

Then within Matplotlib, I am also importing the colours and the lines for developing the graphics. I'm just running this code. So just a word of caution.

Sometimes when you are running code in a collab environment, it might take some time depending upon the internet connection and the number of simultaneous processes that are happening. Because this is a shared cloud-based environment, sometimes the processing might be slower. Otherwise, if you are running it on your own desktop computer in a Jupyter notebook, it would be relatively faster.

Now, first let us understand how to use different colours. So colours in Matplotlib can be accessed using different features. First is something called default colour cycle, where colours are defined as strings C0 to C9.

What are these colours? Let us go to the Matplotlib source documentation. Here you can see colours in the default property cycle. For example, if you see the C0 represents TAD blue, all the way C9 represents to Cyan.

What we are trying to understand is how to call this particular colour in our graphic. There are also other ways to call this. There is single letter operation for R for red, G for green, B for blue like that.

Again, you can go to the source and see colour by value. Here are different ways to do that. Colour bar, named colour sequences.

These are named colours. You have BGR for blue, green, red, CMY, K and W. Then you have a palette on which you have different colours and then CSS colours each by different names. You can use all of these by directly recalling their names.

HTML colour names or hexadecimal codes like hash, 2e, 8b57 or something similar. You can call the colours. The point is if you are familiar with the basic colours and how to recall them, you should be able to get across most of the functions.

Like trying to colour the different categories of a bar graph in different colours. Remember in last class, we tried to plot a bar graph of apples and try to arrange different fruit production by their colours. You can simply use the basic colours and get the task done.

However, if you want to customise, build more shades or create more aesthetic features into your plot, you might need different shades of colours. That's when you can use the extended features. I would ask you to refer to the original source documentation so that you can pick up the most appropriate colour for your job.

But the most important part is you need to know how to access the colour. Then which code you have to use for the colours. It is more important to know how to access them.

Then you can always try to modify the colours that you want. There is no point in trying to remember for all the colour palettes that you have. But if you know this is the function of this particular colour palette, that would suffice.

In our example, how do we demonstrate the use of different colours? We are trying to plot a graph which is nothing but a straight line. It is a straight line and in each plot, we will try to vary the colour and certain attributes of the graph so that you will be able to appreciate the use of different colours. What I am trying to do is trying to plot a figure.

I use the plt.figure and I am fixing the figure size. Here also I will demonstrate one thing. You have the figure size.

You can alter the figure size. You can also make it larger. This will be a larger figure.

Let me just go back to the original. You can alter the size of the figure and you will get the different sizes of the figure. I am plotting the different lines.

I am first plotting the red colour line. I am plotting a straight line 0 to 1. Y-axis is 0. X-axis is 0 to 1. Then you have colour. For colour, I am choosing red which is in R. I am labelling it as red because I have chosen red colour.

Line width is 4. You can also alter the line width. I am just drawing this easy line. Let me showcase the change in the colour feature.

If I am making the figure size to be 10 x 5, this is how it would look. It would look a little broader. You can alter the size of the figure by modifying this.

To this, now I am adding one more line. It is a bit above. See, automatically this has shown some interesting features.

Let us try that. I have just used the tab button to accept the suggestion given by the intelligent agent. It is showing the three colours RGB, each with line width.

Then it is labelling them automatically as RGB. The point was here we are not using the title and label. That is why it is not available.

Once I add that colour specifications, legend left centre box, box has to be anchored to one point. Then I am asking plot to be shown. Now you have these labels also that have come.

You are able to clearly see which colour is representing what. Let us run the code that we intended to do. In this case, I am using six plots.

I am using the six different lines, each one representing a different colour schema so that you would know how to use the different colour schemas. In the first, I am using colour to be indicated by single letter. Then you have the default colour panel C0 to C1.

I am using C1. Then you have calling the colour by its own name like lime green. Then colour by hexadecimal.

Then colour with varying ranges between 0 to 1. It is a RGBA tuple. You get the colours. Then you have the greyscale 0 to 1 where 0 is black, 1 is white.

Depending upon the transparency, it gives a greyscale. Let us run this code. Now you can see, you have six lines, each one using a different colour, using different colour specification or using different colour function.

First is simply calling it by a single letter R, G or B. Then you have C0 to C9. Then you have the lime green or directly calling the name. Then you have the hexadecimal code.

Then you have the RGBA tuple, in which you can proportionately colour red, green, blue. Then you have the greyscale which is 0 is black, 1 is white and in between is the varying shades. Let me show this particular feature.

If I were to make it 0.2 then I can run this or this to be C8. I am just running. Do not worry about the name.

If you can see, this has become more darker compared to the earlier. We could do this. Sorry for this.

This was the original specification. I am just copying it up. Then again pasting it here.

I am changing this to C7. This to 0.1. Let it be 0.5 here. Can you see the difference now? First of all, the second line has become C7.

This has become greyish. It is no more orange because you have used C7 instead of C1. We have modified the RGB tuple.

We have changed the aspect ratio. We have got a lighter shade here. Though it is still kind of a blue, we have a lighter version of blue.

The final grey line has become darker because you have reduced the transparency. It is 0.1 only. Like that you can use different colour schemes to recall different colours onto the graph and colour your geometric objects.

Again, I suggest you look at the colour palettes that are there in the examples of source documentation and try to experiment with them so that you know which colour scheme to use for your graphic. Now let us go to using colour maps. These are different ways to recall the colours onto your graphic.

They are especially useful for colouring heat maps or using the colour based on a third variable that is not there in the plot. For example, if you are plotting a scatter diagram like this, which shows the relations between X-axis and Y-axis, but you can colour the markers in the scatter plot using a third variable maybe indicating the intensity of correlation between these two variables or depending upon a third parameter. If you are talking about height and weight, then you can talk about age or income distribution.

You can talk about some other third variable which is represented by the colour. In this case, using a colour map is very useful. We will just quickly run this example and try to see how to use the colour map.

The colour map is recalled by two features. It is VIR, IDIS and then you have Z function. These two are available for drawing the colour maps and you can also customise your colour map using this particular function, which I would leave as an exercise for you.

Now, let us quickly see how to run the colour map. We are just trying to understand the use of a particular colour map. What are we doing? We are doing colour map is VIRDIS.

Then you have the colour map of Z. Rest all is just the plotting function that you have and we are generating the plot. Run the selection. This is what you are getting.

VIRDIS is more balanced than Z because it is sometimes non-uniform. The point here is when you have a distribution and you want to show a gradual colour saturation, it is better to use a colour map and you can recall the function using the VIRDIS or Z functions. You can also talk about percentage achievement of target across regions and then you can colour shade them with lowest being red and highest being green.

You can do all this using VIRDIS function. I am also using another example where I am drawing a scatter plot, but I am using the custom C map. You can customise the colour map that you want to use.

You can use custom C map. I am using red, yellow, green and then we are plotting the scatter plot. Simply run this.

You will get a scatter plot where the colour schema is customised. The strength of relationship is shown using three colours, somewhere between red, green and yellow, varying shades of this showing the strength of relationship and scatter plot with a custom colour map. Here the main point is you are using a segmented colour map from the list using the matplot.colours, linear segmented colour map and then you are giving the list of colours.

If I add cyan, let me see whether this will work any differently. Run the selection. Just that I have added cyan, now I have cyan colour also coming in and naturally the order in which you give the colours is which it is taking.

Let us also try this out. Instead of adding cyan at the end, I am trying to add cyan at the beginning and trying to see what will happen. See, the way in which you order colours is the way in which it takes.

So cyan is below, red is there. You can also rearrange the sequence of colours in which you want to show the relationship using the custom colour map. This is also available here.

You can select individual colours in a map. You can experiment with this from the source documentation. I seriously encourage you to do so.

Let us move to the next decorating feature of graphs which is line and marker styles. We use line diagrams quite often in our graphics to show changing over time or you want to establish a relationship. You use lines or there is a scatter plot.

You use markers to understand how two or three variables are associated with each other in the same graphic. In this case, you might have to use the preattentive attributes to distinguish lines between categories or between markers. You need to colour them or present them in different size and shape.

Go back to the gestalt principle. The colour of common objects is natural for the human eye to treat objects belonging to the same colour, belonging to the same group or same shape belonging to the same group. We can get those features attributes onto your visualisation using these features.

Let us try to see how one can control colours of the lines and markers in a plot. First, I am trying to draw two lines simply. Line customisation.

First, I am preparing the data. It is nothing but the way I am creating a linear data from 0 to 10. X variable is 0 to 10 evenly spaced at 20 points.

Then, you have y is equal to x. It is a straight line through origin. Then, you have another y line which is an intercept to y is equal to mx plus c. Here, c is 2. This is the data that I have generated. Simple data.

Once again, I am explaining the data. X is 0 to 10. Then, y is equal to x. It is a straight line through origin.

Then, you have y2 which is x plus 2. It is nothing but the straight line through origin line shifted by two points on the y axis with an intercept of 2. What I am doing? I am just plotting the graph. It should just look like a straight line through origin. Figure is equal to axis.

Plt dot subplot is used for the plotting function. Then, x dot plot, axe dot plot for plotting x comma y. Then, I am labelling the series as series 1. You simply have the 0 to 10 x. y is equal to x. It is a straight line through origin. Simple.

It is just blue in colour. Now, what I am trying to do? I am trying to add the second plot to this particular data series. How can I do this? I can simply go back.

This. Use the same function. Instead of y1, I will add y2.

Label it as series 2. Run the entire thing. Now, you have two lines. By default, Python or the Matplotlib has coloured these two series as different.

Now, you want to change the line style for the plot in series 1. Instead of a straight line, you wanted to have a dotted line. This is one more way to show that you are customising the series. You are customising the series by using a different shape or the geometrical object Instead of line, you are having a dotted line.

We are getting attributes like colour. I want to colour it to be in red and line style to be double dash. Once you get into these line styles, there are many options here.

Go back to the source. You have lines, bars, markers. Then you have the entire different type of examples in which you can customise lines.

Or the markers for that matter. You have line plot. You can then talk about how to change the characteristics of lines.

Different line styles. How to do a dashed dot? How to do the spread out dot? Continuous, densely dashed dotted? All these things are available. You can look at these features on how to plot this.

I am just running this. What has happened? First, line has become coloured in red and line style is double dash. You can see the difference.

Label is series 1. Next, I am trying to change the series 2. In this, my objective is I am adding a marker. Marker size is 10. Colour is green.

Line style is done. On Y2, I am adding a marker which is a star. Marker size is 10.

I think it is quite large. I am just going to make it 5. Just running this. Now you have the markers on Y2.

At each point, you have markers indicating the points on the Y axis. You can see the difference between Y1 and Y2 series very clearly. I am going to add a legend.

It is showing show legend. This is one interesting use of the intelligent agent. It suggests you what to do next.

You have the plot. You have the legend here. In fact, I can make it more.

If you see, there is one small difference. I thought I will show you. I am adding a title.

Finally, you have the marker and line customisation. This is how you can customise your lines and markers. Again, please look into the examples that are there in source documentation.

Kindly practise them. Again, you need not remember everything how to use a command. But you should know where to look when you need to do something.

That is the main catch. You cannot remember beyond the basic commands. But you should know if you are able to do the basic commands and get the basic colours.

Then you can start customising depending upon the case. In that particular instance, you want to use a dashed dot or a loosely dotted line or a long dash with an offset. You can go to the particular example, take down the code and use it in your particular visualisation.

Now, let us talk about line styles. What are the line styles? You can control the thickness, dash patterns and the cap styles. Cap styles indicate the ends of the lines, whether they are arrow or projecting.

All these things can be done using line styles. I am just again going to take a very simple example here. I am trying to showcase.

I am just copying this. We will give you the source sheet so that you can practise and learn how to do. What are we doing? We are just doing an np array.

y base is equal to np array of 1, 2, 3, 2, 1. It will take values 1, 2, 3 and 2, 1. That is how y will look like. We are plotting the function. We are using the different line styles.

What are we doing? In the first, we are plotting y base, which is simply the existing. We just plotted the y base. This is how it is looking.

It ranges from 0, goes up to 3, then comes back. This is how it is 1, 2, 3, 2, 3, 1. This is how it is going for all x that is ranging from 0 to 4. Now, line style is like this. Label, we have given dashed.

Then, what we are doing? We are shifting the y plot by one number. Here, if you see, it is 2. I just wanted to run this 2 together. Now, you have 2 series.

y is equal to 0. Then, it starts from y is equal to 2. Then, we have one more line. We are just adding one more line. This is how the three lines look like.

Can you see the difference? In the first case, you are using double dash. Then, it is dashed dot. Then, it is semicolon.

Let me see if it works with two semicolons. There is no thing called two semicolons. This is not going to work.

Be careful about what line styles you give. You have to be aware of which line styles are acceptable in Python. In all these three cases, line width is same.

Now, I am going to vary the line width. I am going to make 4. I am going to make this 5. Run this again. You can see the line width is also changed.

You can use all these to distinguish the series. Finally, I am also using the dash cap style is equal to round. You get the different arguments that you can put in the plot.

This is a lot of documentation. You can also experiment from this particular pop-up box. This is how it will come.

If you run the only line, it shows the rounded end points. Now, let us run the 4 together. I will just put this to be 7. This is how it is going to look like.

See the lines are straight ended here, but they are round here. Let me run the entire thing together. The problem here is Y line is cutting at 5. Let me put it till 8 and then run.

You have the entire legend coming clearly and all lines being shown in your graph. The point here is you can customise the shape of line, your markers as in the last example. All these can be used to distinguish various attributes on your graphic.

Coming to customising spines. Spines are lines that form the border of the plotting area. Spines are nothing but the borders of your plotting area.

In our example, spines would be these 4 lines. What should you do? First, you have to get the current axis object with this function. Axe is equal to plotGCA and axis each spine.

Top, bottom, left, right. There will be 4 spines via axis.spines. We will see this. Then you can use the methods like setVisible, setFalse, setLineBridge, setColor.

Let us discuss each one of these. How can you distinguish? You just want to switch off. You want to only show the X and Y axis, but not the top border and the left border.

You can setVisible is equal to false for these 2 spines and only highlight these 2. Or else you just want to colour the X axis. Then you can set colour for particular spine i.e. bottom to be a particular colour. Let us look at an example.

These things will become clear. First, we are trying to plot a very simple function. We are going to get Y is equal to 0, 1, 4, 9. Since it is nothing but a parabolic function, Y is equal to X square.

0 square 0, 1 is 1, 2 is 4, 3 is 9 and 4 is 16. I am using plt.plot for plotting that. I got this function.

Let me extend this to 25, 36. Just for fun. This is how we get.

As you increase, it will become smoother. I am just getting the current axis using this function. Axe is equal to plot.gca. Sorry.

Again, in examples, you have the axis, axis property, broken axis. All this you can also use from your examples. Just I am trying to show how to delve deeper into the finer aspects of Mathplotlib and Decorative Graphs.

So, what we can do? The object is to make the top and right spines invisible. Now, you have this top. Let me again tell, this is the top, this is the bottom, this is the left, this is the right.

The task is to make the top and right spines invisible. I am just saying, axe.spines in the bracket, which particular spine I am talking about? I am talking about the right spine. Then I am using the function setVisible is equal to false.

Which means, it will not show. Otherwise, it default, it will show. Unless you use setVisible set to false, it won't be invisible.

I am just running this function. You see, the top and the right spines have disappeared. Now, I want to colour the remaining ones and also change the line widths.

I am accessing the left spine. I am accessing the left spine and setting the line width of the spine to be 4. Let me put it to be 6. Then, I am also saying, the left spine should be in green colour. I am now making it red.

Then, let the bottom spine be with line width of 4. Instead of dark blue, I am using lime green. I am just using lime green. You can see, how can you customise the widths of your spine, change colour, make them visible or invisible.

Now, I will finally do this. Doing all this, left and bottom. What should you expect? You should expect, all the spines should go and only this should come.

----------End----------